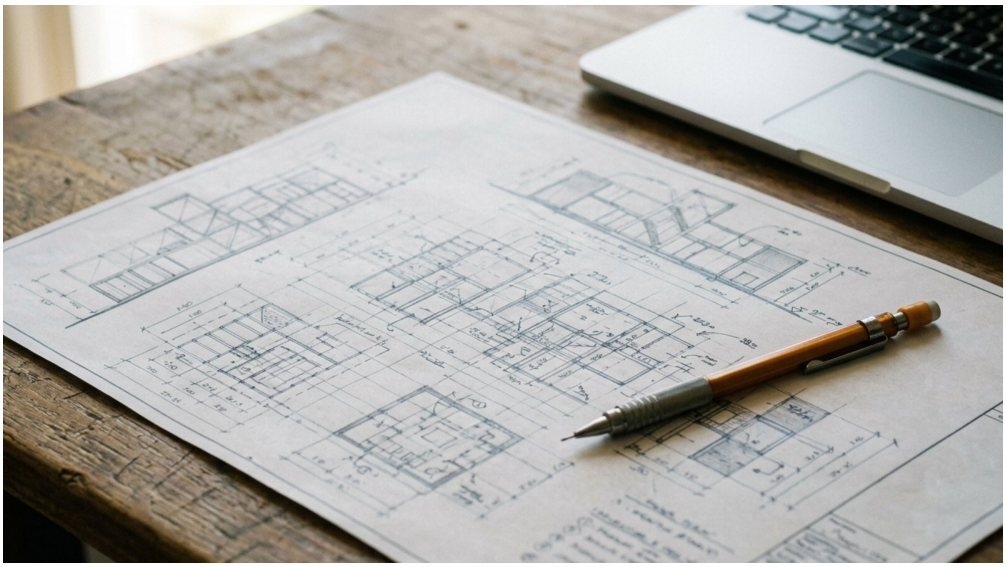
Code Review for AI-Assisted R Package Development

Ronald G. Thomas

2026-08-18



A design authored by a human and a package implemented by an assistant still needs one owner who can defend every line: this is the checklist for becoming that owner.

Introduction

An increasingly common development pattern pairs a human architect, who designs package structure, API, and behavior, with an AI assistant that writes the implementation and the test suite. The arrangement is productive, but it creates a responsibility gap. The human author's name goes on the package, and with it the obligation to understand, maintain, and defend every line of code, regardless of who typed it.

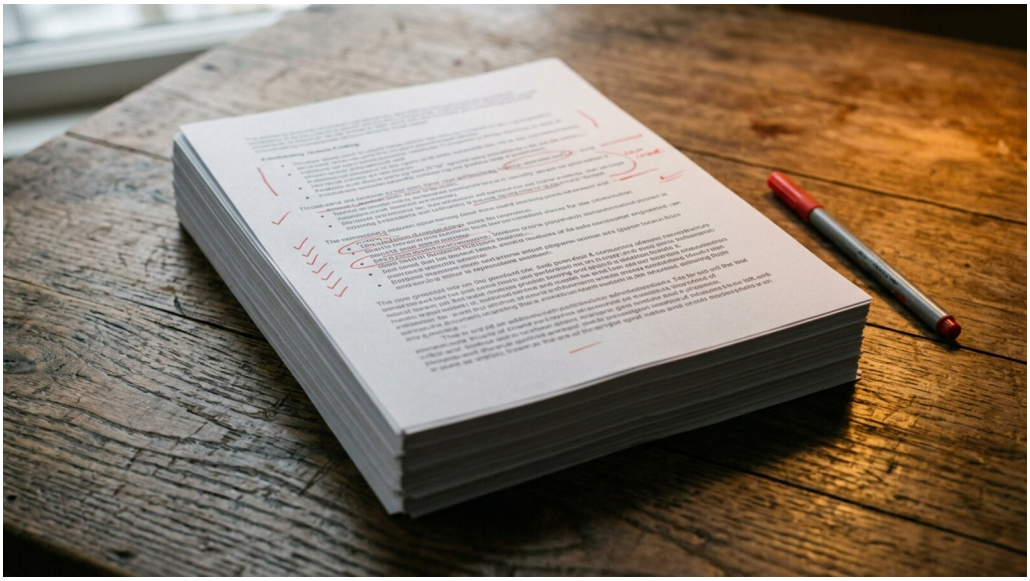
This post presents a methodological framework for closing that gap: a structured code review process for R packages developed through human-AI collaboration. It synthesizes established code review practice with considerations specific to AI-generated code, and it ends with an explicit ownership test rather than a vague sense that the review "went well."

Motivations

- **The responsibility gap is real, not rhetorical.** An author who cannot explain a function's implementation cannot debug it, extend it, or defend it to a CRAN reviewer.
- **AI-generated code has characteristic failure modes.** Over- engineering, hallucinated functionality, and pattern mimicry do not show up the same way in human-written code, so a generic review checklist misses them.
- **A generated test suite is not independent evidence.** When the same assistant writes the implementation and the tests, both share the same misconceptions, and passing tests carry far less information than they appear to.
- **Coverage metrics are a floor, not a review.** A package can reach 100% line coverage while every assertion checks nothing meaningful.
- **Ownership needs a definition, not a feeling.** "I reviewed it" and "I can rebuild this from memory" are different claims, and only the second one licenses shipping the package under your name.

Objectives

1. Define the five review phases (structural, function-by-function, test suite, integration/system, security) and what each phase is for.
2. Catalog the AI-generated code patterns that deserve specific scrutiny during function review, with a detection method for each.
3. Provide a concrete note-taking and severity-classification system for tracking findings through remediation.
4. Separate the checks a machine can perform from the checks only the author can perform, and show how large the second set really is.
5. Establish a verifiable test for "ownership": what an author must be able to do, unaided, before claiming the package as their own.



What Is This Framework For?

The scope is deliberately narrow. It addresses review of R packages where a human author defined requirements, architecture, and design decisions, an AI assistant wrote the implementation code and tests, and the author now seeks to establish complete ownership and understanding of the result before it ships. It is not a general code review guide, and it does not replace `R CMD check` or CRAN's own policies; it sits alongside both.

A thorough review in this context serves five purposes: comprehension (the author understands every implementation detail), correctness (the code does what it claims), quality (it meets ordinary maintainability standards), security (it introduces no new vulnerabilities), and consistency (the implementation matches the stated design, rather than drifting toward whatever the assistant found convenient).

How Much Review Is Proportionate?

The full process described below is disproportionate for a package of four functions that only its author will ever run. Scale the effort to the risk, which is roughly the product of three factors: how many people will run the code, how bad a silently wrong answer would be, and whether the package touches personal, clinical, or otherwise sensitive data.

Tier 1, personal tooling. A package you alone use, whose wrong answers you would notice immediately. Run Phase 1 and Phase 2, plus the automated checks. Skip the formal annotation template; skip the integration phase unless functions actually compose. The ownership test still applies, because the cost of not understanding your own tools is paid later regardless of audience size.

Tier 2, shared research software. A package used by collaborators or published to CRAN, whose wrong answers would propagate into someone's analysis before anyone noticed. Run all five phases and the full checklist. This is the tier the rest of this post assumes.

Tier 3, sensitive or consequential. A package touching identifiable human-subjects data, clinical decision support, or any result that would be expensive to retract. Everything in Tier 2, plus: the ethics and data-handling items, an independent reviewer who is not the author, and adversarial testing of the numerical core against a reference implementation. Self-administered review is not sufficient at this tier, and the framework's central limitation, that the author grades their own understanding, becomes disqualifying.

Two notes on tiering. First, a package moves up tiers over time without anyone deciding that it has; the trigger is usually the first external user. Re-tier at each release rather than at first publication. Second, tiering governs how much of the checklist you run, never whether a failed item may ship unfixed. A "no" is still a blocker at every tier.

Prerequisites

Before starting, assemble the original design documents and requirements, the package `DESCRIPTION` and `README`, any conversation logs or prompts used during development, existing test results and coverage reports, and `R CMD check` output. Then set up a working environment:

```
# Install the package in development mode
devtools::load_all()

# Generate fresh documentation
devtools::document()

# Run initial quality checks
devtools::check()
rcmdcheck::rcmdcheck()
```

Static analysis tooling worth having installed before the review starts: `lintr` for style and likely-error detection, `covr` for coverage measurement, `goodpractice` for a broader quality assessment, `cyclocomp` for complexity metrics, and `pkgcheck` for the rOpenSci conformance checks discussed later in this post.

The conversation logs deserve particular emphasis. They are the only record of what was actually asked for, and the only way to distinguish a function that implements a design decision from a function that implements something the assistant inferred. Keep them; they are the provenance trail this framework depends on.

The Five Review Phases

Phase 1: Structural Review

Begin at the package level, before looking at any individual function.

File structure. Confirm the package follows standard R package conventions (`DESCRIPTION`, `NAMESPACE`, `R/`, `man/`, `tests/testthat/`, `vignettes/`, `inst/`, `data/`).

Dependency audit. For every entry in `Imports`, ask whether it is actually necessary, whether it belongs in `Suggests` instead, whether its version constraint is appropriate, and whether it introduces a security or maintenance concern of its own.

Export surface. Read the `NAMESPACE`. Are only the functions that are meant to be public actually exported? Is the API surface as small as the design allows, with internal helpers kept private?

Phase 2: Function-by-Function Review

This is the core of the process. For every exported and internal function, work through five questions in order:

Correctness. Does the function produce the expected output for representative inputs? Are edge cases handled? Does it fail with an informative error rather than a cryptic one?

Design alignment. Does the implementation match the intended design, or has the assistant introduced patterns that were not requested?

Input validation. Are argument types checked where it matters? Are bounds enforced? Do the default arguments make sense?

Documentation quality. Does the roxygen2 block describe actual behavior? Are all parameters documented, is the return value specified, and do the examples run without error?

Code clarity. Can you explain what each line does and why? Are names descriptive and consistent across the file?

For each function, read the documentation before the implementation, then trace the happy path with typical valid inputs and at least one failure path with an invalid or edge-case input. Verify that the test suite actually exercises the behavior just traced, and deliberately question any assumption the code makes implicitly.

Work in dependency order, from the bottom of the call graph upward. Reviewing a caller before its callee means holding unresolved questions about the callee in memory while reading the caller, and then re-reading the caller once the callee is understood.

Patterns specific to AI-generated code

These patterns recur often enough in assistant-written code to warrant a dedicated pass. Each is given with what it looks like, how to find it, and what to do about it.

Over-engineering. Abstraction or generality beyond what the requirements called for: a configuration argument with five options where the design specified one behavior, a class hierarchy where a list would do, a plugin mechanism nothing plugs into.

Detection: compare the function's parameter list against the design document. Any parameter you cannot trace to a stated requirement is a candidate. `cyclocomp` will flag the resulting complexity, though it cannot tell you the complexity was unnecessary.

Response: delete the unused generality. It is not free; every untaken branch is code you have promised to maintain and must understand to claim ownership.

Defensive redundancy. Checks or fallbacks that duplicate a guarantee already made elsewhere: re-validating an argument that the calling function already validated, a `tryCatch()` around an operation that cannot throw, `if (is.null(x))` after `x` has been assigned unconditionally.

Detection: for each guard, ask what input would trigger it. If you cannot construct one, it is redundant.

Response: remove it, or, if the guard encodes a real invariant, replace it with an explicit assertion that documents the invariant. Redundant guards are worse than useless because they suggest the invariant is uncertain when it is not.

Pattern mimicry. Code that looks correct because it resembles a familiar idiom, while missing a requirement specific to this package. The classic form is a function that follows the shape of a tidyverse verb without honoring the grouping semantics the shape implies.

Detection: this is the hardest to catch mechanically, because the code reads well. The reliable method is to state the requirement in your own words first, then read the implementation and ask whether it implements that requirement or merely the idiom.

Response: rewrite from the requirement rather than patching the idiom.

Inconsistent style. Naming or structural variation across functions that a single author would not have introduced: `n_obs` in one function and `num_observations` in the next, mixed assignment operators, argument order that varies between functions taking the same inputs.

Detection: `lintr` catches operator and formatting inconsistency. Naming inconsistency needs a manual pass over the full export list, which is why the API surface is reviewed as a whole rather than function by function.

Response: pick one convention and apply it everywhere. This is cheap to fix before release and expensive after, because fixing it later is a breaking change.

Hallucinated functionality. References to functions, arguments, or packages that do not exist. Note that this fails loudly when the call is executed and silently when it is not, which means it hides in error branches, `\dontrun{}` examples, and documentation prose.

Detection: run every example including the escaped ones; execute every error path in tests. For prose references, check each named function actually exists with `exists()` or by opening the help page.

Response: fix the reference, then look for others in the same region. Hallucination clusters.

Fabricated dependencies. A distinct and more dangerous case of the above: the assistant suggests a package that does not exist. Published measurements put the rate of non-existent package suggestions from large language models at roughly one in five, and attackers have begun registering the names that models hallucinate most often, so that a later install of the "missing" dependency retrieves attacker-controlled code. This is a supply-chain vulnerability, not a typo, and it is treated as such in Phase 5.

Detection: verify every entry in `Imports` and `Suggests` against the registry it claims to come from, and check the publication date and publisher of anything unfamiliar. A package registered last month that exactly matches the name an assistant suggested is a red flag, not a convenience.

Response: remove it, and pin what remains.

Fabricated citations. References in vignettes, documentation, or comments to papers, DOIs, or specifications that do not exist, or that exist but do not say what the text claims.

Detection: resolve every DOI and URL. For each cited method, read enough of the source to confirm it describes the algorithm actually implemented.

Response: correct or remove. A fabricated citation in a research software package is a scholarly integrity problem, not a documentation defect.

Comments that describe intent the code does not implement. Distinct from hallucinated functionality, and harder to see, because the comment reads as authoritative and the code below it is syntactically fine. A comment claiming a loop handles the empty case, above a loop that does not.

Detection: read comments as claims to be verified, not as guidance. Where a comment states a behavior, construct the input that would exercise it.

Response: fix the code if the comment is the requirement; delete the comment if the code is correct. Never leave both.

Scope drift across a session. In long generation sessions, later functions silently contradict decisions made earlier: a null-handling convention established in the third function is abandoned by the twelfth.

Detection: pick two or three cross-cutting conventions (missing value handling, error signaling, return type on failure) and check every function against each one, rather than checking each function against everything.

Response: pick the convention that is right, not the one that is more common in the file, and apply it throughout.

Plausible but unjustified numerical constants. Tolerances, convergence criteria, iteration caps, seeds, and thresholds that look reasonable and have no stated basis.

Detection: grep for numeric literals in the `R/` directory. Every one that is not obviously structural needs a comment explaining where it came from.

Response: justify, cite, or parameterize. An unexplained `1e-8` is a design decision nobody made.

Outdated patterns. Deprecated functions or superseded approaches that were correct at some point in the assistant's training data but are not the current recommendation. In R this has a concrete and checkable form: rOpenSci treats `RCurl`, `rjson`, `RJSONIO`, and `XML` as obsolete outright, and `sp`, `rgdal`, `maptools`, and `rgeos` as superseded, and a model trained on years of published R code will suggest exactly these.

Detection: `pkgcheck` flags all eight by name. Beyond dependencies, check for superseded idioms within packages that are themselves current.

Response: migrate. The migration is only going to get more expensive.

Correctness traps in generated R code

Separately from the patterns above, a set of R-specific mistakes appear often enough in generated code to be worth a dedicated sweep. Most are drawn from *R Packages* by Wickham and Bryan and from the CRAN *Repository Policy*, and several are policy violations rather than matters of taste.

Top-level code in `R/`. This is the most consequential item in the list and the one most likely to be present, because it is the signature of code written in a script idiom. *R Packages* puts it plainly: any R code outside a function is suspicious and should be carefully reviewed. The reason is that

code in `R/` runs when the package is **built**, not when it is used, and the build often happens on a different machine at a different time. Three concrete failures follow:

- A path computed at top level with `system.file()` is baked in at build time and points somewhere that does not exist on the user's machine.
- An object computed at top level captures the capabilities of the build machine, which is typically a headless server. A cached colour table computed at build time will report that the terminal has no colours, forever, on every user's machine.
- An alias written as `foo <- pkgB::blah` freezes the definition of `blah()` as it existed on the build machine. If a bug in `blah()` is later fixed, your package keeps the broken version until it is rebuilt. The correct form is `foo <- function(...) pkgB::blah(...)`.

The fix in every case is the same: push the code down into the body of a function so it executes on the user's machine at run time.

Modifying the R landscape. A package function should not change global state that outlives the call. `library()` and `require()` alter the search path and should never appear below `R/`; dependencies belong in `DESCRIPTION`. `source()` has no legitimate use inside a package. `options()`, `par()`, `setwd()`, `Sys.setenv()`, `Sys.setlocale()`, and anything touching the random number generator must be restored on exit. The current idiom for this is the withr package (`withr::local_options()`, `local_par()`, `local_dir()`, `local_envvar()`, or the general `withr::defer()`), which captures the old value and schedules restoration for you. Base R's `on.exit()` also works, but you must capture and restore the state yourself, and you must remember `add = TRUE` or your second `on.exit()` silently discards the first.

Depending on the user's landscape. The mirror image of the above. Sorting is the common case: `sort()` and `order()` depend on the collation locale, so results differ across platforms. Use `method = "radix"` where order must be stable.

ASCII-only source. CRAN requires that `.R` files contain only ASCII characters; non-ASCII must be written as `\uxxxx` escapes. The usual offenders are curly quotes that arrive through copy and paste, which is exactly how generated text tends to arrive. `tools::showNonASCIIfile()` finds them.

Suggested packages used unconditionally. A package in `Suggests` may not be installed. Every use of one below `R/` must be guarded with `requireNamespace("pkg", quietly = TRUE)` or `rlang::check_installed()`, with either a clear error or a documented fallback. Generated code routinely calls a suggested package as though it were a hard dependency.

Namespace and S3 registration. Prefer `pkg::fun()` over importing; import into your namespace only for operators, for functions used very heavily, or in a tight loop. A method for a generic that your package neither owns nor imports needs `@exportS3Method pkg::generic`, not `@export`; plain `@export` will silently fail to register it. An unregistered method works under `devtools::load_all()` and fails once installed, which is the one configuration interactive development never exercises.

Side effects at load. If a package genuinely needs setup, it goes in `.onLoad()`, with `.onAttach()` reserved for startup messages, which must use `packageStartupMessage()` so users can suppress them. Never disturb the random seed there; `withr::with_preserve_seed()` exists for this.

File system discipline. Nothing may be written outside the session temporary directory, and even there it must be cleaned up. Persistent user data belongs in `tools::R_user_dir()`, kept small and actively managed. Nor may a package launch external software during examples or tests

without closing it, which makes clipboard access doubly problematic: the clipboard counts as the user's home filespace and on Linux may spawn a process.

Numerical and data handling.

- Comparing doubles with `==` rather than a tolerance. Prefer `expect_equal()` over `expect_identical()` in tests, because numerical precision varies across platforms.
- Missing-value policy left implicit; `NaN` and `Inf` handled incidentally rather than deliberately.
- Reference semantics. Generated code usually assumes copy-on-modify throughout, which is false for `data.table`, `R6`, and environments: such a function mutates the caller's object.
- `data()` calls to load the package's own lazy-loaded datasets. These create objects in the user's global workspace, potentially overwriting what is there, and are unnecessary.

Phase 3: Test Suite Review

Coverage numbers are a starting point, not a conclusion:

```
coverage <- covr::package_coverage()
covr::report(coverage)
```

Before assessing the tests, be clear about what a generated test suite can and cannot tell you. **If the same assistant wrote both the implementation and the tests, the tests are not independent evidence of correctness.** They were derived from the same understanding of the requirement, so any misreading of the requirement is present in both, and the test will confirm the misreading rather than catch it. A green test suite in this situation demonstrates internal consistency, which is worth something, and correctness, which it does not demonstrate at all. This is the single most important thing to understand about reviewing AI-assisted packages, and it is why Phase 3 is a review of the tests themselves rather than a reading of their results.

The practical consequence: for the behaviors that matter most, write at least one test yourself, from the requirement, without looking at the implementation. If it passes, that is evidence. If the generated suite already covered the same behavior, compare the two tests and ask why they differ.

With that established, for each test file separately assess whether the assertions check substantive behavior or merely that the code runs without erroring, whether boundary conditions are tested, whether expected failures are verified rather than only successes, whether tests are independent of one another's side effects, and whether a reader can tell what each test is meant to validate.

Then check correspondence in both directions: every documented behavior should have a corresponding test, and every test should correspond to a documented or intended behavior. Tests that check conditions no longer relevant to the design are as much a maintenance liability as missing tests.

Two notes on coverage itself. First, 100 percent is not the goal. *R Packages* is explicit that going from 90 or 99 percent to 100 is often not the best use of effort, because the last few lines usually demand awkward gymnastics or new mocking machinery, and a test suite whose maintainability was sacrificed to a number is a bad trade. Direct effort at the code that is tricky, not at the code that is uncovered. Second, and pulling the other way, coverage counts lines executed, and a

generated suite executes a great many lines while asserting very little, so a high number in this setting is weaker evidence than the same number would be elsewhere.

Further checks that coverage will never surface:

- **Run the tests in random order.** Order dependence between tests is common in generated suites, because the assistant writes each test in the context of the ones before it. A suite that passes only in file order is not a suite.
- **Count the skips.** Skipped tests are close to invisible in continuous integration, so it is entirely possible to believe a suite is passing when much of it never ran. Read the skip summary in the check output deliberately, and confirm each skip is one you intended.
- **Read every snapshot before accepting it.** Accepting a snapshot because the test framework offered to accept it is the coverage fallacy wearing different clothes: it records what the code does, not what it should do. Note that snapshot tests are skipped on CRAN by default.
- **Check that test files are almost entirely `test_that()` calls.** The rule for `R/` applies again here: top-level code in a test file has an awkward "test file scope" that makes tests depend on each other invisibly and makes a failing test hard to reproduce in isolation. Setup belongs inside the test, in a helper function, or in `helper.R`/`setup.R`. Some duplication between tests is the correct price for this; test code should be obvious rather than dry.
- **Check that tests do not attach packages.** `library(somePkg)` in a test changes the search path, so the tests no longer run under the conditions the package will actually be used in. Use `pkg::fun()`, exactly as below `R/`.
- **Check what the tests write and where.** Only the session temp directory, and cleaned up afterward. `withr::local_tempfile()` and `withr::local_tempdir()` handle this. Fixture files belong under `tests/testthat/` and should be reached with `test_path()`, with the script that generated them kept alongside.
- **Guard tests that touch the network or the clock.** Tests calling an API need `skip_on_cran()`, and ideally recorded fixtures. Tests depending on `Sys.time()` or the current date will fail on a day you are not watching.
- **Do not test timing or parallelism.** Check machines are heavily loaded and run package checks concurrently, so elapsed-time assertions and assumptions about available cores are flaky by construction. Flakiness is not only your problem: your package is checked whenever a package you depend on updates, so a flaky test can delay someone else's release.
- **Check that a test's fixture is what the function will really receive.** A test that constructs its own input and then asserts on it is only as good as that construction.

Phase 4: Integration and System Review

Trace data flow across function boundaries rather than within a single function. Do the functions compose correctly when chained? Are intermediate data structures consistent from one stage to the next? Identify and evaluate any state management: global variables or options, environment modifications, file system operations, external connections. Verify error propagation: are errors caught at the right level, do the messages carry enough context to act on, and is cleanup performed correctly when something fails partway through?

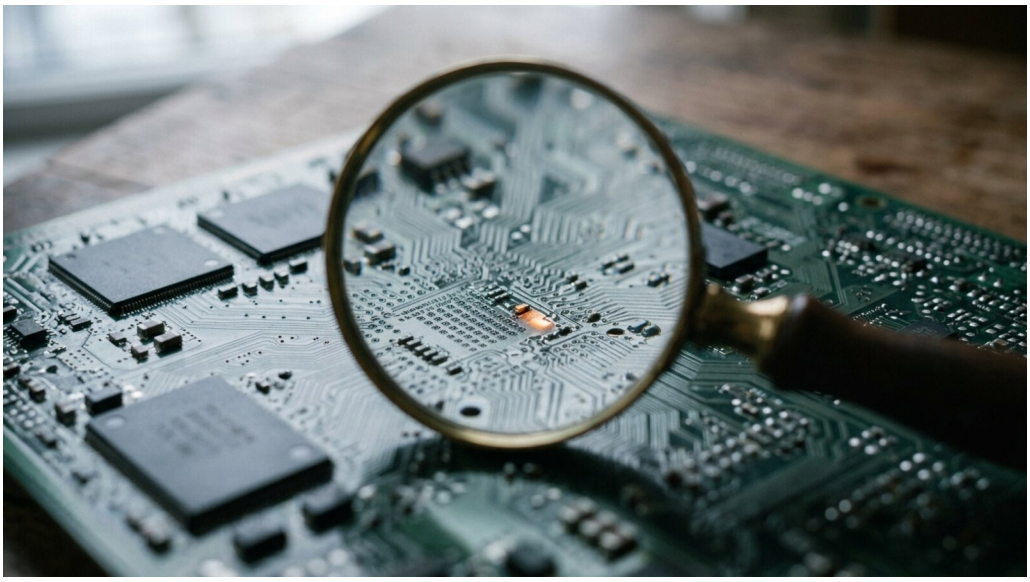
Phase 5: Security and Safety Review

Examine every user-facing input for the standard vulnerability classes: unsanitized input used in file paths, input passed to system commands, query construction, and serialization or deserialization of untrusted data. For network and file system operations, confirm connections are closed, temporary files are cleaned up, and URLs are validated before use.

Credentials. Credentials should be read from environment variables or `keyring`, never passed as a documented function argument, and never printed in any message, warning, or error, including in verbose mode. Recorded HTTP fixtures and test cassettes must be inspected and redacted before their first commit. Check the git history as well as the working tree: a secret committed and later removed is still in the history, and tooling exists for this (`gitleaks`, `trufflehog`, or GitHub's own secret scanning) rather than relying on inspection.

The dependency chain. Are the dependencies actively maintained, is the chain as small as the design allows, and does anything in it have a known vulnerability? Be aware that R's tooling for the last question is genuinely weak compared to other ecosystems: there is no equivalent of `npm audit`. What exists is `oysteR`, which queries the OSS Index, and `riskmetric`, which scores packages on maintenance signals rather than vulnerabilities. Neither is a substitute for reading the dependency list and knowing what each entry is for. This is a real limitation of the R ecosystem and it should be stated plainly rather than papered over with a tool name.

Fabricated and squatted dependencies. As described in Phase 2, assistants suggest non-existent packages at a measurable rate, and those names are a known target for pre-registration by attackers. Every dependency should be verified to exist in the registry it claims to come from, with a publisher and registration date that make sense for a package of its stated age. Pin versions in a lockfile and verify hashes; an unpinned dependency resolved at install time is a dependency someone else can change.



Documentation Review

User-facing documentation and developer documentation get separate passes. The `README` should state the package's purpose clearly, give installation instructions, include a basic usage example, and point to further documentation. Vignettes should build without errors, present coherent workflows with realistic examples, and complement rather than duplicate the function-level documentation. Internal documentation should explain any non-obvious algorithm, record the reasoning behind design decisions, and document the contribution process if the package expects one.

Annotation and Severity

Maintain structured notes during the review rather than relying on memory. A minimal per-function template:

```
## Function: function_name

### Status: [Reviewed | Needs Changes | Approved]

### Understanding
[Summary of what the function does, in your own words]

### Concerns
- [Issue 1]
- [Issue 2]

### Questions
- [Question for further investigation]

### Changes Required
- [ ] Change 1
- [ ] Change 2
```

The "Understanding" field is the load-bearing part, and it is worth seeing filled in, because the discipline it enforces is not obvious from an empty template. Here it is for a small function that normalizes a numeric vector:

```
## Function: rescale_unit

### Status: Needs Changes

### Understanding
Maps a numeric vector onto [0, 1] by subtracting the minimum and
dividing by the range. Takes `x` (numeric) and `na.rm` (logical,
default TRUE). Returns a numeric vector of the same length. If all
elements are equal the range is zero, and the function returns NaN
for every element rather than erroring.
```

```
### Concerns
- The all-equal case returns NaN silently. The design document says
  degenerate input should error. This is a design-alignment failure,
  not a bug in the arithmetic.
- `na.rm = TRUE` as the default contradicts the convention set in
  `summarize_group()`, which defaults to FALSE. Scope drift.
- Comment above the range calculation claims zero-range input is
  "handled"; it is not.

### Questions
- Is NaN or an error the right behavior for a constant vector? The
  design says error, but the two callers both tolerate NaN.

### Changes Required
- [ ] Error on zero range, with a message naming the argument
- [ ] Change `na.rm` default to FALSE for consistency
- [ ] Delete or correct the misleading comment
- [ ] Add tests for constant input and all-NA input
```

Note what the exercise surfaced: three of the four findings came from writing the summary, not from reading the code. A function you cannot summarize this way has not been reviewed, whatever the checklist says.

Classify every finding by severity: **Critical** (incorrect behavior, a security vulnerability, or a data-corruption risk), **Major** (a significant deviation from requirements or poor error handling), **Minor** (style inconsistency, a documentation gap, or a small inefficiency), or **Enhancement** (a suggestion beyond the current requirements). The severity tier determines the order of remediation, not whether an issue gets fixed at all.

The boundaries are where reviewers disagree, so an example of each:

- **Critical.** A summary function drops rows containing `NA` without documenting it, so every downstream count is wrong and nothing signals it. Silent wrongness is critical even when the code never errors.
- **Major.** A file-reading function assumes UTF-8 and fails with an uninformative error on any other encoding. It fails loudly, so nobody is misled, but the package is unusable for a class of input the design said to support.
- **Minor.** One function documents its return value as "a data frame" where the others give the columns. Real, cheap, and nobody is harmed by shipping it next week instead of today.
- **Enhancement.** The function could accept a grouped data frame in addition to a plain one. Nothing in the requirements asks for it.

The dividing line between Critical and Major is not severity of consequence but whether the failure is silent. Between Major and Minor it is whether the package still does what it promised.

Remediation and Final Verification

Work through issues in severity order: critical issues affecting correctness or security first, then major issues affecting usability or maintainability, then minor issues and enhancements as time allows. For each fix, understand the root cause before touching the code, and implement the fix yourself rather than delegating it back to the assistant without understanding it. Add or modify a test to prevent regression, and update documentation if the observable behavior changed.

After remediation, re-run the full test suite and `R CMD check`, confirm coverage has not decreased, and review the diff in version control before considering the issue closed. Before calling the review complete, run the full verification pass:

```
# Full check, as CRAN would run it
devtools::check(cran = TRUE)

# Test coverage
covr::package_coverage()

# Additional static analysis
lintr::lint_package()
goodpractice::gp()

# rOpenSci conformance checks
pkgcheck::pkgcheck()

# Reverse dependencies, if anything depends on this package
revdepcheck::revdep_check()
```

Then test installation in a clean environment (`remove.packages()` followed by `devtools::install()`) and run every example in the documentation with `devtools::run_examples()`.

Packages containing compiled code need more than this. `R CMD check` will not find a missing `PROTECT`, an out-of-bounds write, or undefined behavior that happens to be benign on your compiler. Run the package under the sanitizer and valgrind containers that `rhub::rhub_check()` provides, and use `rchk` for the R-specific protection analysis. Generated C and C++ is particularly prone to protection errors, because the pattern of `PROTECT` and `UNPROTECT` pairs is idiomatic enough to mimic and strict enough that mimicry is not sufficient.

When the review fails

The checklist says a "no" is a blocker. It is worth being explicit about what that means in practice, because the tempting response is to note the issue and ship anyway.

There are three legitimate responses to a function you cannot bring yourself to approve, and only three. **Rewrite it yourself**, from the requirement, using the generated version as at most a reference. **Cut the feature**, if the function exists to serve a requirement you are no longer sure you need. **Do not ship**, if neither of the first two is possible right now.

What is not on the list is asking the assistant to fix it. That produces a function you understand no better than the one you started with, and it converts a known problem into an unknown one. If the assistant's help is genuinely needed, the sequence is: understand the root cause yourself first, decide on the fix yourself, and only then use the assistant to type it. The distinction is not pedantic. It is the whole difference between a package you own and a package you host.

Relation to Established Review Standards

This framework was not developed in a vacuum, and it is worth being explicit about where it overlaps with existing peer review practice and where it adds something. The relevant comparison is rOpenSci, a nonprofit founded in 2011 that runs an open peer review process for scientific R packages. Its development guide, rOpenSci Packages: Development, Maintenance, and Peer Review (source at github.com/ropensci/dev_guide), is the closest thing the R ecosystem has to a consensus statement of what a package review should cover. Much of it is mechanized in pkgcheck, which runs automatically on every submission, so the guide is not merely aspirational: a substantial fraction of it is enforced before a human reviewer sees the package. rOpenSci maintains a separate and stricter set of statistical software standards for packages implementing algorithms rather than accessing data.

Beneath the review standards sit the two documents that define what a correct R package is in the first place: the CRAN *Repository Policy*, which is binding on anything distributed through CRAN, and *R Packages* by Wickham and Bryan, which is the field's standard explanation of why the rules are what they are. A large share of the checklist below comes from these two rather than from the review standards, and they are the source of most of the items that generated code actually fails. That is not a coincidence: the review standards mostly govern the shape of a package, which is easy to get right, while the policy and the book govern its behavior under conditions the author never sees, which is where code written from a script idiom breaks.

Two other standards are worth naming. The JOSS review checklist is language-agnostic and weighted toward research impact and provenance rather than code internals. The Turing Way's checklist for the code review process is the most general of these and the only one that treats licensing and legal compatibility as a first-class review concern.

The checklist in the next section merges the items from all of these into the five-phase structure, so the rest of this section is about positioning rather than content.

What the external standards do not cover

All three assume an *external* reviewer. Someone other than the author reads the code, and the author's own understanding of their package is taken for granted rather than tested. This framework inverts that assumption, because in AI-assisted development the author's understanding is precisely the thing in doubt.

That is not a rhetorical distinction. Reading the full development guide at its current revision, there is no guidance anywhere on AI-assisted development: no mention of generative models, large language models, or machine-generated code, in any of its chapters or templates. The consensus R standard is, at present, silent on the development pattern this post addresses. Its checks are all conformance checks, and conformance is exactly the property AI-generated code satisfies most

easily and least meaningfully. A package can pass every mechanized check while its author cannot explain a single function in it.

External review practice has begun to move in this direction elsewhere. JOSS now asks reviewers to check for transparent disclosure of generative AI use, and for evidence of sustained development over months rather than a repository generated rapidly and recently. Those are provenance questions of exactly the kind the Establishing Ownership section below argues an author should be able to answer about their own work.



The Consolidated Review Checklist

This is the whole checklist, in one place. Copy it per package rather than trying to hold it in memory across several reviews at once, and scale which sections you run to the risk tier established earlier.

Items ending in a tool name are checked mechanically; run the tool rather than reading for them. **Items with no tool name are the ones only you can do, and they are the majority.** That distribution is the argument of this post in one glance: the automatable half of review is large, well-specified, and already solved by other people's software, and it is not the half that establishes ownership.

> **i** This is not a CRAN conformance checklist
>
> A number of items below come from the CRAN *Repository Policy*, and the sections that follow include the policy requirements most often broken by generated code. That does not make this list a substitute for the policy, and completing it is neither necessary nor sufficient for acceptance.
> Three reasons. The policy is revised without announcement, so any list is a snapshot of a moving target. Parts of it are matters of judgment rather than rules: "no warnings or significant notes" is decided by a person, and CRAN enforces expectations that are not written

down anywhere. And the actual mechanical gate is `R CMD check --as-cran` run against current R-devel on every platform, which catches things no enumeration will reach.
This list is also deliberately wider than CRAN in the directions that matter here (ownership, the AI failure modes, ethics, test independence) and narrower on submission mechanics, which are out of scope. It overlaps CRAN policy; it does not contain it. For a list maintained specifically against the policy, use ThinkR's *Prepare for CRAN*, and read the policy itself before you submit.

Phase 1: structural

- ☐ File structure follows standard R package conventions `pkgcheck`
- ☐ `NAMESPACE` generated by roxygen2, not hand-edited `pkgcheck`
- ☐ No nested `R/` subdirectory, no `renv` infrastructure committed inside the package `pkgcheck`
- ☐ No scrap files (`.DS_Store` and similar) tracked `pkgcheck`
- ☐ Every `Imports` entry is necessary, correctly tiered (`Imports` vs `Suggests`), and version-constrained appropriately
- ☐ `NAMESPACE` exports only what is meant to be public
- ☐ Every exported name traced to a design requirement

Phase 2: function-by-function

- ☐ Every exported and internal function reviewed for correctness, design alignment, input validation, documentation quality, and code clarity
- ☐ Functions reviewed in dependency order, not file order
- ☐ Happy path and at least one failure path traced for each function, with the test suite confirmed to exercise both
- ☐ Every parameter traced to a stated requirement; unrequested generality removed
- ☐ Every guard clause has a constructible triggering input

Phase 2: AI failure modes

- ☐ Over-engineering: unused generality deleted
- ☐ Defensive redundancy: unreachable guards removed or converted to documented assertions
- ☐ Pattern mimicry: requirement stated in your own words before reading the implementation
- ☐ Inconsistent style: naming, argument order, and assignment operator uniform across the export list `lintr`
- ☐ Hallucinated functionality: every referenced function and package confirmed to exist, including in error branches and escaped examples
- ☐ Fabricated dependencies: every dependency verified in its registry, with a plausible publisher and registration date
- ☐ Fabricated citations: every DOI resolved and every cited method confirmed to describe what was implemented
- ☐ Misleading comments: every comment read as a claim and verified
- ☐ Scope drift: cross-cutting conventions checked across all functions, not within each
- ☐ Unjustified numerical constants: every non-structural literal justified, cited, or parameterized

Phase 2: R correctness traps

- ☐ No R code at top level in `R/` beyond object creation; nothing whose value depends on the build machine (`system.file()` paths, capability probes, `pkg::fun` aliases)
- ☐ No `library()`, `require()`, or `source()` below `R/`
- ☐ `options()`, `par()`, `setwd()`, `Sys.setenv()`, `Sys.setlocale()`, and RNG state restored on exit, preferably with `withr::local_*()`; any `on.exit()` uses `add = TRUE`
- ☐ Nothing written outside the session temp directory, and cleaned up; persistent user data only via `tools::R_user_dir()`
- ☐ No external software launched without being closed; no clipboard access in examples or tests
- ☐ No `set.seed()` inside package functions; parallel code sets `RNGkind("L'Ecuyer-CMRG")`; load hooks preserve the seed
- ☐ Sorting is not locale-dependent where order must be stable
- ☐ `.R` files are ASCII only, with `\uxxxx` escapes for anything else `tools::showNonASCIIfile()`
- ☐ Every use of a `Suggests` package below `R/` guarded by `requireNamespace()` or `rlang::check_installed()`
- ☐ External functions called as `pkg::fun()` unless there is a reason to import; whole-namespace `@import` avoided
- ☐ S3 methods for foreign generics registered with `@exportS3Method pkg::generic`, and the package tested as installed rather than only via `load_all()`
- ☐ Setup confined to `.onLoad()`; startup messages in `.onAttach()` via `packageStartupMessage()`
- ☐ Reference-semantics objects (`data.table`, `R6`, environments) not mutated through arguments
- ☐ Nothing modifies code or bindings in base, recommended, or other packages' namespaces
- ☐ Nothing in the examples or tests could be read as malicious or anti-social, which the policy prohibits in those words
- ☐ No floating-point equality comparison; tolerances used and justified
- ☐ Missing-value policy explicit per function; `NaN` and `Inf` handled deliberately
- ☐ No `data()` calls for the package's own lazy-loaded datasets

Phase 3: test suite

- ☐ Coverage report reviewed file by file, not just as a percentage `covr`
- ☐ At least one test per critical behavior written by you, from the requirement, without reading the implementation
- ☐ A sample of test bodies read to confirm assertions check substantive behavior, not just that the code runs
- ☐ Each behavior tested in one place, and the external interface tested rather than the internals
- ☐ Every documented behavior has a corresponding test, and every test corresponds to a documented or intended behavior
- ☐ Test files consist almost entirely of `test_that()` calls; shared setup lives in helpers, `helper.R`, or `setup.R`
- ☐ No `library()` or `source()` below `tests/`
- ☐ Test suite passes in randomized order
- ☐ Skip count read and every skip confirmed as intended
- ☐ Every snapshot read before acceptance
- ☐ Tests write only to the session temp directory and clean up (`withr::local_tempfile()`); fixtures reached with `test_path()`, with their generating script kept
- ☐ Tests touching the network or the clock guarded, and backed by recorded fixtures where possible
- ☐ No assertions about elapsed time or available cores
- ☐ `expect_equal()` used rather than `expect_identical()` for anything numeric

- ☐ Whole suite runs in about a minute; anything longer is `skip_on_cran()`
- ☐ What remains after the CRAN skips still exercises every feature of the package. Policy allows long-running tests to be made optional; it does not allow the checks that remain to stop covering the package
- ☐ Zero-length, single-row, all-`NA`, and unsupported-type inputs tested rather than assumed to error

Phase 4: integration and system

- ☐ Data flow traced across function boundaries, not just within individual functions
- ☐ State management (globals, options, environment modifications, file system, external connections) identified and evaluated
- ☐ No function assigns to the global environment; data-importing functions return objects
- ☐ Error propagation verified: caught at the right level, with actionable messages and correct cleanup on partial failure

Phase 5: security, secrets, and supply chain

- ☐ Every user-facing input checked against the standard vulnerability classes (unsanitized paths, shell commands, query construction, deserialization)
- ☐ Network and file system operations confirmed to close connections, clean up temporary files, and validate URLs
- ☐ Credentials read from environment variables or `keyring`, never a documented argument
- ☐ No credential printed in any message, warning, or error, including verbose mode
- ☐ Recorded HTTP fixtures inspected and redacted before first commit
- ☐ Git history scanned for secrets, not just the working tree `gitleaks`
- ☐ Tests requiring credentials skipped on CRAN and handled for pull requests from forks
- ☐ Dependency chain reviewed for maintenance status and known vulnerabilities `oysteR`, `riskmetric`
- ☐ Dependency versions pinned and hashes verified

API surface as a whole

- ☐ Argument names and orderings consistent across exported functions; for functions returning the type they consume, the data object is the first argument so the function composes with `|>`
- ☐ One naming scheme applied uniformly
- ☐ Exported names checked for collisions across CRAN, not just base R `pkgcheck`
- ☐ Console output uses `message()`, `warning()`, or `cli`, never `print()` or `cat()` outside `print.*()` and `str.*()` methods
- ☐ Verbosity controlled by an option or environment variable rather than a function argument
- ☐ No package startup message except where genuinely required

Documentation

- ☐ Every exported function and every exported dataset documented `R CMD check`
- ☐ Every function documents its return value under `@return`, which CRAN requires for a first submission; a placeholder restating the function name does not count `pkgcheck`
- ☐ Every parameter's default stated in the documentation text, in the form "a logical value (default `TRUE`) determining …"

- ☐ Every exported function has at least one runnable example, and not all of them wrapped in `\dontrun{}` `pkgcheck`
- ☐ Examples throw no errors, avoid network access, and complete well inside the ten-minute limit
- ☐ Examples use only packages in `Imports` or `Suggests`, and attach or qualify them, since examples run in the user's environment
- ☐ `\dontrun{}` and `\donttest{}` used only where necessary, and those examples still executed in CI via `--run-dontrun` and `--run-donttest` `pkgcheck`
- ☐ Internal functions marked `@keywords internal`; developer-only exports marked `@export` plus `@keywords internal`
- ☐ Related functions cross-linked and grouped with `@family`
- ☐ `Title` in title case with no trailing period, software names in single quotes, no "A package for …" opening, acronyms expanded in `Description`
- ☐ URLs valid, including redirects, which CRAN rejects `urlchecker`
- ☐ Spelling checked mechanically `spelling`
- ☐ Vignettes substantive rather than perfunctory; reference index grouped by topic
- ☐ Vignette and data sizes within CRAN limits, and packaged data documented

Licensing and attribution

- ☐ `LICENSE` present, CRAN- or OSI-accepted, consistent with the `DESCRIPTION License` field `pkgcheck`
- ☐ Licenses of all dependencies compatible with the package's own, remembering that compatibility is not symmetric: MIT code can be bundled into a GPL package, but not the reverse
- ☐ Bundled or adapted third-party code attributed, with original authors given `ctb` and copyright holders `cph` in `Authors@R` with a comment saying what they wrote, or recorded in `inst/COPYRIGHTS`; `LICENSE.note` present if bundled components carry different licenses
- ☐ `LICENSE.md`, if present, listed in `.Rbuildignore`, since CRAN does not accept the full text of standard licenses
- ☐ Any generated code closely resembling an identifiable existing implementation traced and attributed, or rewritten. Two wrinkles make this sharper for AI-assisted code than it looks. Stack Overflow answers are CC BY-SA licensed, which is compatible only with GPLv3, and models are trained heavily on Stack Overflow. And a derivative work does not require copied text: re-implementing an algorithm from a source can qualify even when no line is shared
- ☐ Copyright holders tracked from the start, since relicensing later requires permission from every one of them
- ☐ The chosen license grants CRAN the right to distribute the package in perpetuity
- ☐ Any change of license flagged explicitly in the submission
- ☐ Trademarks respected, including in the package name
- ☐ The maintainer's warranty honestly satisfiable. CRAN requires the maintainer to warrant that they act on behalf of all credited authors and have their agreement, and that copyright over every component is clear and unambiguous. This is the provision AI-assisted development strains hardest, because it is a claim about the origin of code whose origin the author cannot inspect. CRAN says nothing about generative tools; it simply asks you to warrant something they make harder to know

Dependencies

- ☐ Dependency count sane relative to comparable packages; `pkgcheck` flags the top five percent of CRAN by number of `Imports` `pkgcheck`
- ☐ No obsolete dependency: `RCurl`, `rjson`, `RJSONIO`, `XML` outright, and `sp`, `rgdal`, `maptools`, `rgeos` as superseded `pkgcheck`
- ☐ Functions imported from the package where they are defined, not one that re-exports them
- ☐ `Imports` used rather than `Depends`; test and documentation packages in `Suggests`
- ☐ Minimum version constraints justified individually
- ☐ No `Remotes` field at submission time, and no dependency pinned to a version unavailable from CRAN or Bioconductor
- ☐ Internet-using code fails gracefully with an informative message when a resource is unavailable, rather than erroring or warning during check
- ☐ No more than two cores used simultaneously
- ☐ All downloads use secure mechanisms; rate limits respected so that checks do not provoke 429 or 403 responses, remembering that other packages may be hitting the same resource
- ☐ No dependency that restricts users' rights, since that restriction propagates to your users
- ☐ System dependencies declared and checked in `configure`

API evolution

- ☐ Removed arguments retained in the signature to error explicitly, never absorbed silently by `...`
- ☐ Renamed functions aliased and warned about before removal
- ☐ Deprecation staged through `.Deprecated()` then `.Defunct()`, with corresponding man pages
- ☐ Semantic versioning, and a `NEWS.md` entry for every user-visible change `pkgcheck`
- ☐ Reverse dependencies checked before any breaking change `revdepcheck`

Verification beyond one machine

- ☐ `R CMD check --as-cran` clean on Linux, macOS, and Windows `rhub`
- ☐ Tested against R release, oldrel, and devel `rhub`
- ☐ CI running check, tests, coverage, and lint on every commit `pkgcheck`
- ☐ Clean-environment install tested, not just `load_all()`
- ☐ Compiled code run under sanitizers, valgrind, and `rchk` `rhub`
- ☐ Numerical results checked for stability across platforms and BLAS implementations

Project-level documentation and provenance

- ☐ Statement of need: what problem the package solves, for whom, and how it compares to existing packages
- ☐ `CONTRIBUTING` and a code of conduct present `pkgcheck`
- ☐ `DESCRIPTION` carries `URL`, `BugReports`, and a maintainer `pkgcheck`
- ☐ `CITATION` present, using `bibentry`, citing any wrapped data source or algorithm as well as the package `pkgcheck`
- ☐ ORCID recorded for authors who are people, ROR for organizations `pkgcheck`
- ☐ Example datasets checked for licensing and inclusivity
- ☐ Issue and pull request trackers in a state a newcomer can read
- ☐ AI involvement in development documented

Ethics and data handling

- ☐ Data sources' terms of use, `robots.txt`, and rate limits respected in code, examples, and vignettes, and linked from the documentation
- ☐ Handling of personally identifiable or reasonably re-identifiable data identified, with de-identification and secure-storage workflows documented and demonstrated
- ☐ Where the package touches vulnerable populations or human-subjects data, informed-consent practice supported and documented; accessibility of data does not imply consent to its use

Package contents, if you intend to distribute

- ☐ No binary executable code in the source package
- ☐ Source provided, or provided in an easily convertible form, for every component covered by an open-source license
- ☐ Source tarball within 10MB, and no single data or documentation component over 5MB, or an exemption argued for
- ☐ Data compression chosen deliberately rather than left at the default `tools::checkRdaFiles()`
- ☐ Java sources, if any, in a top-level `java/` directory

Submission process, if you intend to distribute

- ☐ `Maintainer` is one named person with a working address, not a mailing list or a role account
- ☐ Maintainers of reverse dependencies notified at least two weeks before an update that could break them
- ☐ Version number increased
- ☐ Submission spaced sensibly from the last one; CRAN asks for one to two months between updates once a package is established
- ☐ Any remaining check notes explained in `cran-comments.md` rather than left for the reviewer to interpret

Before shipping: ownership sign-off

- ☐ Can explain any function's purpose and implementation without referring to the code
- ☐ Can predict the function's behavior for a novel input
- ☐ Can identify where a new requirement would require a change
- ☐ Can debug an issue without AI assistance
- ☐ Can respond to a user's issue, review an external contribution, and extend the package's functionality unaided
- ☐ Passed the cold-start, modification, and teach tests described in Establishing Ownership
- ☐ Willing to state a two-year maintenance commitment, or name a succession plan

A "no" anywhere in this checklist is a blocker, not a note for later. What that means in practice is set out in "When the review fails" above, and what each unchecked box costs if it ships unresolved is set out in Establishing Ownership below.

What the Tools Can and Cannot Check

The checklist above is long, and a fair reading of it is that most of it is someone else's problem. That is true, and worth making explicit, because the residue is the point.

Tool	Covers
`R CMD check --as-cran`	Package structure, documentation completeness, example execution, encoding, many CRAN policies
`pkgcheck`	rOpenSci conformance: `@return` tags, examples, CITATION, CONTRIBUTING, license, URLs, CRAN-wide name collisions, obsolete and excessive dependencies, CI and coverage presence
`goodpractice`	Cyclomatic complexity, line length, `T/F` shorthand, `sapply()` use, dependency count, common defects
`lintr`	Style, assignment operators, naming conventions, likely errors
`covr`	Line coverage, file by file
`urlchecker`, `spelling`	Dead links, redirects, misspellings
`revdepcheck`	Breakage in packages that depend on yours
`rhub`	Multi-platform, multi-R-version checks, sanitizers, valgrind
`gitleaks`, `trufflehog`	Secrets in the git history
`oysteR`, `riskmetric`	Dependency vulnerabilities and maintenance signals, partially
`tools::showNonASCIIfile()`, `checkRdaFiles()`	Non-ASCII source characters, data compression

What no tool in that table can check: whether the implementation matches the design, whether a test asserts something that matters, whether a comment is true, whether a numerical constant was chosen or guessed, whether generality is warranted, whether a cited paper says what the vignette claims, and whether the author understands any of it. Every one of those is a judgment, every one of them is where AI-generated code fails, and every one of them is unmarked in the checklist above.

There is also a category that falls between the two, worth naming because it is easy to assume the tools have it covered: several CRAN requirements are enforced by a human reader rather than by `R CMD check`. The conventions for `Title` and `Description`, the requirement that every function document its return value on a first submission, the prohibition on writing outside the session temp directory, the rule that internet-using code must fail gracefully rather than error during check. These are checkable in principle, mostly unchecked in practice, and a package can pass every automated check and still be rejected on them.

Run the tools first, always. They are fast, they are free, and clearing them removes the noise that would otherwise obscure the judgments. Just do not mistake a clean report for a completed review.

Things to Watch Out For

- **Coverage as a proxy for quality.** A test that asserts nothing meaningful still counts toward the coverage percentage.

- **A green suite written by the same assistant as the code.** It demonstrates internal consistency, not correctness. Treat it as a regression net, not as evidence.
- **Delegating the fix back to the assistant.** Understanding the root cause and then asking the assistant to patch it is not the same as understanding the fix; do the remediation yourself.
- **Treating hallucinated functionality as a rare edge case.** It surfaces often enough in assistant-written code that a dedicated existence check for every referenced function and package is worth the time, and in the case of dependencies it is a supply-chain control rather than a tidiness measure.
- **Reviewing functions in file order instead of dependency order.** Reviewing a caller before its callee means re-reading context twice; work from the bottom of the dependency graph up.
- **Skipping the "why," not just the "what."** A function can be correct and still misaligned with the stated design; design alignment is a separate question from correctness.
- **Conflating "reviewed" with "understood."** The annotation template's "Understanding" field is not decorative; if you cannot fill it in from memory, the function has not actually been reviewed.
- **Accepting a snapshot because the framework offered to.** The snapshot records what the code does. You are reviewing whether that is what it should do.

Lessons Learned

Conceptual understanding:

- Ownership is a testable claim, not a feeling of familiarity.
- A generated test suite is not independent evidence about generated code, because both encode the same reading of the requirement.
- Coverage and correctness are different axes; a review needs both.
- AI-generated code has a distinct failure signature (over-engineering, mimicry, hallucination, scope drift) that a generic checklist will not catch.
- The automatable half of package review is large and already solved; the residue is entirely judgment, and the residue is the review.

Technical:

- `covr::package_coverage()` plus `covr::report()` gives a navigable, file-by-file coverage view, not just a percentage.
- `goodpractice::gp()`, `lintr::lint_package()`, and `pkgcheck::pkgcheck()` each catch a different class of issue than `R CMD check`, and are worth running separately.
- A clean-environment install/reinstall test surfaces dependency issues, and unregistered S3 methods, that a `devtools::load_all()` session will never expose.
- Running the test suite in randomized order surfaces inter-test dependencies that no amount of coverage will reveal.

Gotchas:

- High coverage can mask assertion-free tests; read a sample of test bodies, not just the coverage report.

- `R CMD check` passing does not imply the design was followed; that requires the separate design-alignment pass.
- A function's documentation can describe intended behavior accurately while the implementation has drifted from it; verify both directions.
- A comment can be a hallucination too, and it will not fail any check that exists.

Limitations

- The framework is scoped to R packages specifically; the phase structure generalizes, but the tooling references do not transfer to other languages without substitution.
- It assumes the human author already has enough R fluency to evaluate correctness and design alignment; it is not a tutorial on reading R.
- The ownership test is self-administered. The cold-start, modification, and teach tests below make it harder to pass dishonestly, but they remain self-graded, and at the sensitive tier described earlier that is not sufficient: an independent reviewer is required.
- The AI failure modes catalogued here are drawn from current assistant behavior, which changes. The categories have been stable so far, but the frequencies have not, and a pattern that is common today may be rare in a year.
- The framework says nothing about how to review the prompts and design documents themselves, which are upstream of every problem it addresses.

Opportunities for Improvement

1. Guidance on reviewing an AI-assisted *contribution* to an already-owned package, as distinct from reviewing an entire AI-assisted package from scratch.
2. A standard vocabulary for documenting AI involvement in `CITATION.cff`, `DESCRIPTION`, or a `NEWS.md` entry. JOSS now asks for disclosure but does not specify a format, and no R-side convention exists.
3. An empirical base rate for each of the failure modes catalogued here. The categories are observational; nobody has measured how often each occurs per thousand generated lines.
4. Tooling for the checks that are currently manual, particularly comment-versus-code consistency and design-document traceability, both of which look mechanizable.
5. A treatment of the review of prompts and design documents, which this framework takes as given.

Establishing Ownership

Before claiming ownership of the package, verify that you can, without referring to the code: explain any function's purpose and implementation, predict its behavior for a novel input, identify where a new requirement would require a change, and debug an issue without AI assistance. Separately, confirm maintenance readiness: can you respond to a user's issue, review an external contribution, adapt to a breaking change in a dependency, and extend the package's functionality yourself?

Those questions are easy to answer yes to in the moment, immediately after a review, when everything is fresh and the honest answer and the flattering answer are hard to tell apart. Three concrete tests make the claim falsifiable:

The cold-start test. Wait two weeks. Then have someone pick a function at random, or pick one with a random number generator, and write its "Understanding" summary from memory before opening the file. Compare. This is the test the ownership claim actually makes, and the delay is what makes it a test rather than a recital.

The modification test. Implement a small feature you did not plan for, unaided, and time it. The absolute time matters less than whether you knew where to start. If the first twenty minutes went to rediscovering how the package is organized, you have the code but not the design.

The teach test. Write the vignette section explaining the package's central algorithm, without looking at the implementation. Explaining is a stricter test than reading, because it fails on exactly the parts you have been skimming.

Consider also documenting the development process itself: that AI assistance was used, what review process was undertaken, and a clear version-control history showing the remediation work. This is not required by CRAN, but JOSS now asks for it, it is honest, and it gives a future maintainer (possibly a future version of yourself) the context this review process assumed from the start.

One further question, borrowed from rOpenSci's guidance to prospective authors, is worth adding to the sign-off because it is sharper than anything above: **do you expect to maintain this package for at least two years, or to be able to identify a new maintainer?** That is falsifiable and time-bounded in a way that "do you understand it" is not, and it converts ownership from a state into a commitment.

Review does not end at first release. Any AI-generated change should go through the same review before merging, static analysis should run in continuous integration, and older code sections benefit from periodic re-review as the author's own understanding of the package deepens. Knowledge decays without use: working with the codebase regularly, documenting decisions as they are made, and keeping review notes accessible are what keep the ownership claim true over time, not just true at the moment of first publication.

Wrapping Up

What Did We Learn?

Thorough review of an AI-assisted package requires the same rigor a careful reviewer would bring to any package, plus a specific sensitivity to the failure modes assistant-written code tends to produce. The process demands more than a read-through; it requires achieving genuine understanding of every component before accepting responsibility for it. The five-phase structure (structural, function-by-function, test suite, integration, security) gives that process a checklist rather than leaving it to instinct. The ownership test at the end gives it a stopping condition: review is complete when the answers to the ownership questions are honestly yes, not when the reviewer is tired of reading code.

Main takeaways:

- A test suite written by the same assistant as the code is not independent evidence; write the tests that matter yourself.
- Coverage is a floor, not a target; read test bodies, not just coverage percentages.
- AI-generated code has a specific set of failure patterns worth a dedicated review pass, each with its own detection method.
- Most of a package review is mechanizable, and none of the mechanizable part is the part that establishes ownership.
- Ownership is demonstrated by prediction and unaided debugging, not claimed by completing a checklist.

If you are trying this yourself: run every tool in the table before reading a single function, so that the automated noise is out of the way; start with the structural review before touching any function body; keep the annotation template open in a second window rather than trying to hold findings in memory; and do not flip a package to "reviewed" until you can honestly answer every question in the Establishing Ownership section.

See Also

A companion post walks this framework through an applied review of a real package: Exemplar Code Review: The zzobj2fig Package, which uses `zzvim-R` as the review environment and works through structural review, function debugging with `browser()` and `trace()`, and a CRAN-submission checklist that this post does not cover.

Three further companions transpose this same five-phase structure and ownership test onto prose rather than code: Review Methodology for AI-Assisted Blog Post Drafting for blog-length prose, Review Methodology for AI-Assisted Textbook Drafting for book-length, pedagogically structured material, and Review Methodology for AI-Assisted Research Paper Drafting for academic manuscripts, grounded in published journal and publisher AI-disclosure policy.

Key resources:

- R Packages, 2nd ed. — Wickham and Bryan
- Advanced R, 2nd ed. — Wickham
- Writing R Extensions — R Core Team
- CRAN Repository Policy
- rOpenSci Packages: Development, Maintenance, and Peer Review
- rOpenSci Statistical Software Standards
- pkgcheck
- JOSS Review Checklist
- The Turing Way: Checklist for Code Review

Reproducibility

Source document: `docs/CODE_REVIEW_METHODOLOGY.md`, from the `zzobj2fig` package (`~/prj/sfw/10-zzobj2fig/zzobj2fig/`). The five phases, the annotation template, the severity tiers, and the ownership test are taken from that white paper.

Material new to this post: the risk tiers, the detection and response guidance attached to each AI failure mode, the fabricated dependency and fabricated citation modes, the R correctness traps, the test-independence argument in Phase 3, the worked annotation and severity examples, the failed-review protocol, the consolidated checklist, the tooling table, and the cold-start, modification, and teach tests.

Sources consulted, and how. Read from source: the rOpenSci development guide (the `ropensci/dev_guide` repository, all English chapters and templates); *R Packages*, 2nd ed. (the `hadley/r-pkgs` repository, the chapters covering code, testing, dependencies, licensing, documentation, data, and release); and `pkgcheck`'s implemented checks, read as code rather than as documentation. The CRAN *Repository Policy* was read directly. The rOpenSci statistical software standards, the JOSS review checklist, and The Turing Way code review checklist were consulted as published pages rather than source. Everything is cited inline.

AI involvement in this post: drafting and revision of this post were AI-assisted, and the benchmarking against external standards was carried out by an assistant reading the sources listed above directly. Claims attributed to an external standard were checked against that standard's own text rather than a summary of it, at the level of detail indicated above. The argument, the structure, and the framework itself are the author's. This disclosure is included because the post argues that authors of AI-assisted work should make it, and a methodological post that declined to apply its own standard to itself would not be worth much.

Session information:

```
R version 4.6.0 (2026-04-24)
Platform: aarch64-apple-darwin23
Running under: macOS Tahoe 26.6.2

Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib;  LAPACK version 3.12.

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: America/Los_Angeles
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

loaded via a namespace (and not attached):
 [1] compiler_4.6.0  fastmap_1.2.0   cli_3.6.6       tools_4.6.0
```

```
 [5] htmltools_0.5.9 parallel_4.6.0  otel_0.2.0      yaml_2.3.12
 [9] rmarkdown_2.31  knitr_1.51      jsonlite_2.0.0  xfun_0.62
[13] digest_0.6.39   rlang_1.2.0     evaluate_1.0.5
```

Rendered on 2026-09-05 at 18:39 PDT. Source: ~/prj/ryyblog/posts/rp-code-review-methodology/analysis/report/in

---

Let's Connect

Questions, corrections, or a different view on where this framework is too strict or too loose are welcome in the comment thread below.

- **GitHub**: rgt47
- **Email**: Contact form

Related posts in this cluster

This post is part of the *R Package Development and Testing* series. Recommended reading order:

1. Updating an R Package: A Complete Development Workflow
2. Writing a Simple Vim Plugin for REPL Interaction
3. Introducing zzvim-R
4. Testing Data Analysis Workflows in R
5. From testthat to tinytest: Converting an R Package Test Suite
6. **Code Review for AI-Assisted R Package Development** (this post)
7. Exemplar Code Review: The zzobj2fig Package